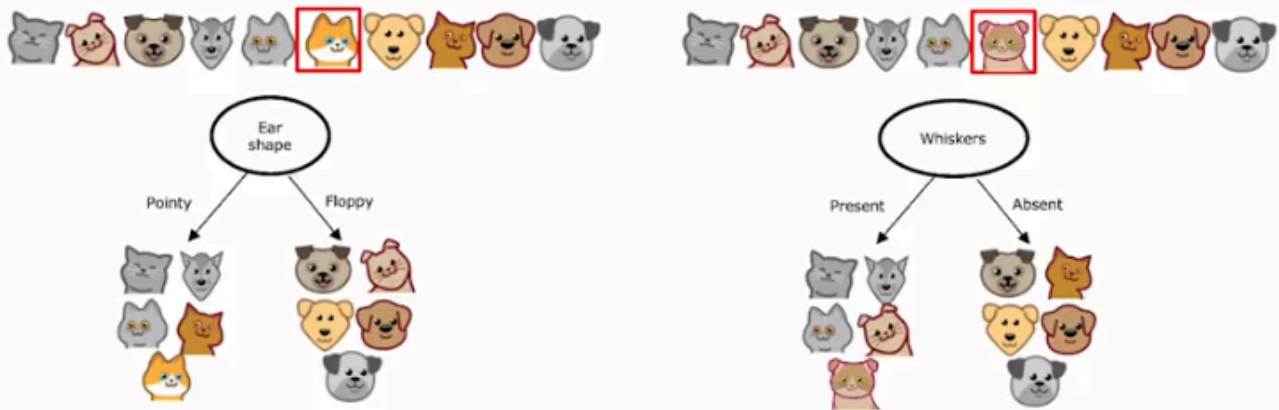


Day 27. Using multiple decision trees

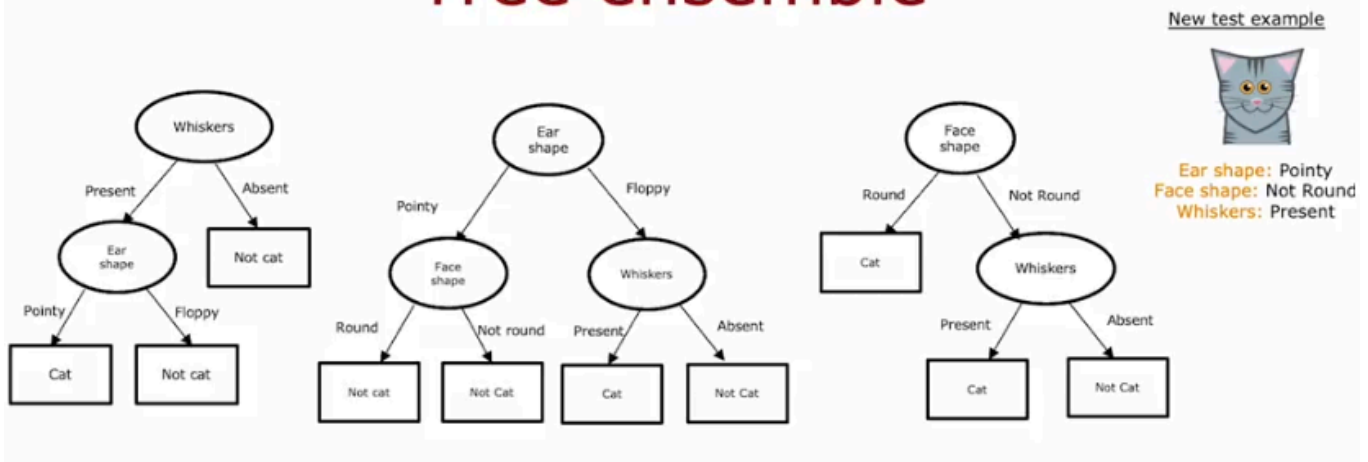
Using a single decision tree might be highly sensitive to small changes in the data. One solution to this is to build a log of decision trees.

Tree ensemble

Trees are highly sensitive to small changes of the data



Tree ensemble



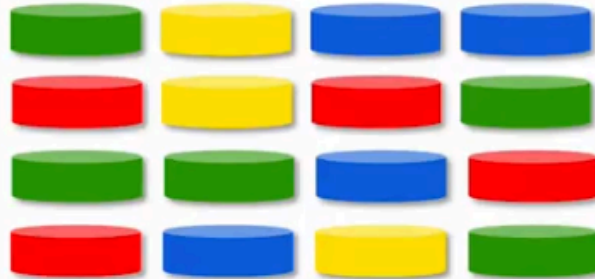
Sampling with replacement

Sampling with replacement

Tokens



Sampling with replacement:



we want to build more datasets that are all slightly different from our original ones.

Sampling with replacement



	Ear shape	Face shape	Whiskers	Cat
	Pointy	Round	Present	1
	Floppy	Not round	Absent	0
	Pointy	Round	Absent	1
	Pointy	Not round	Present	0
	Floppy	Not round	Absent	0
	Pointy	Round	Absent	1
	Pointy	Round	Present	1
	Floppy	Not round	Present	1
	Floppy	Round	Absent	0
	Pointy	Round	Absent	1

Random forest algorithm

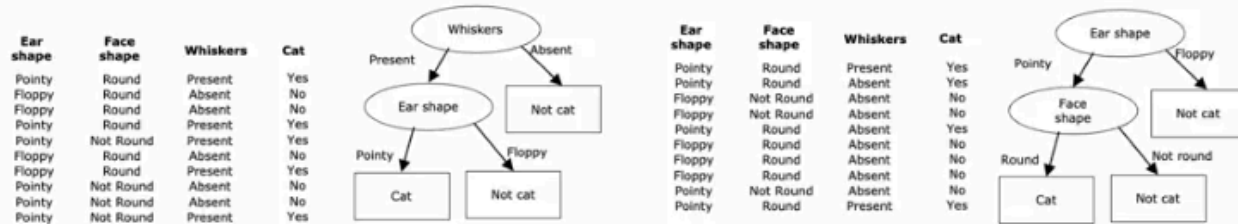
Generating a tree sample

Given training set of size m

For $b = 1$ to B :

Use sampling with replacement to create a new training set of size m

Train a decision tree on the new dataset



this is also called bagged decision tree.

However, it is common to generate trees that splits in the same way at some nodes.

So to further randomizing the features choice, we introduced a little modification.

Randomizing the feature choice

At each node, when choosing a feature to use to split, if n features are available, pick a random subset of $k < n$ features and allow the algorithm to only choose from that subset of features.

XGBoost

Boosted trees intuition

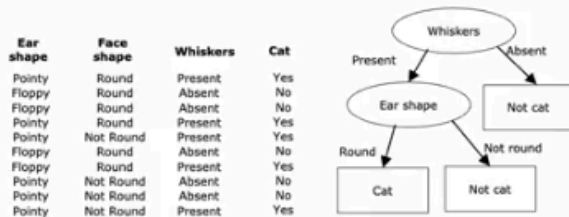
Given training set of size m

For $b = 1$ to B :

Use sampling with replacement to create a new training set of size m

But instead of picking from all examples with equal ($1/m$) probability, make it more likely to pick misclassified examples from previously trained trees

Train a decision tree on the new dataset



Boosted trees intuition

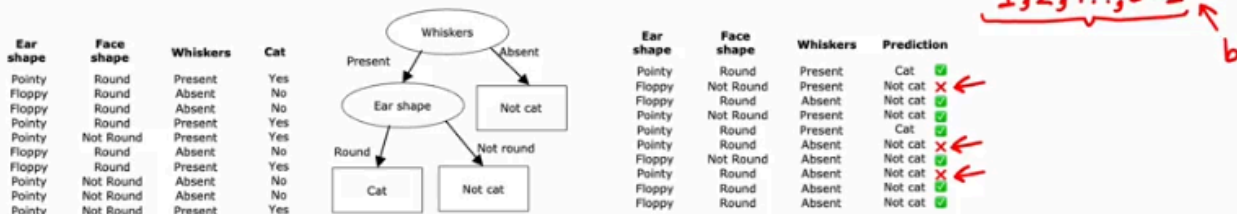
Given training set of size m

For $b = 1$ to B :

Use sampling with replacement to create a new training set of size m

But instead of picking from all examples with equal ($1/m$) probability, make it more likely to pick misclassified examples from previously trained trees

Train a decision tree on the new dataset



XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting
- Built in regularization to prevent overfitting
- Highly competitive algorithm for machine learning competitions (eg: Kaggle competitions)

Using XGBoost

Classification

```
→ from xgboost import XGBClassifier  
  
→ model = XGBClassifier()  
  
→ model.fit(X_train, y_train)  
→ y_pred = model.predict(X_test)
```

Regression

```
from xgboost import XGBRegressor  
  
model = XGBRegressor()  
  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)
```

When decision trees?

Decision Trees vs Neural Networks

Decision Trees and Tree ensembles

- Works well on tabular (structured) data
- Not recommended for unstructured data (images, audio, text)
- Fast
- Small decision trees may be human interpretable

Neural Networks

- Works well on all types of data, including tabular (structured) and unstructured data
- May be slower than a decision tree
- Works with transfer learning
- When building a system of multiple models working together, it might be easier to string together multiple neural networks